

Instituto Politécnico Nacional

Escuela Superior de Cómputo / UPIITA

Ingeniería en Inteligencia Artificial (Plan 2020)

Unidad de Aprendizaje: Bases de Datos

Práctica 1: Modelo Entidad Relación

Alumno: Cesar Javier Martinez Ruiz

Boleta: 2024630871 | **Grupo:** 3BV1

Ejercicio 4. Estado del Arte: Análisis de Tres Artículos Científicos Arbitrados

Ficha Técnica 1: Bases de Datos Vectoriales y Aprendizaje Profundo

1. Cita en formato APA 7.^a edición con DOI

Wang, J., Yi, X., Guo, R., Jin, H., Xu, P., Li, S., Wang, X., Wang, X., Guo, B., & Cheng, L. (2021). Milvus: A purpose-built vector data management system. En *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)* (pp. 2614–2627). Association for Computing Machinery. <https://doi.org/10.1145/3448016.3457550>

2. Problema que aborda el artículo

Aborda la incapacidad de los sistemas gestores de bases de datos relacionales y NoSQL convencionales para indexar, gestionar y consultar eficientemente colecciones masivas de vectores densos de alta dimensionalidad generados por modelos de Aprendizaje Profundo (*Deep Learning*), los cuales requieren cálculos de similitud geométrica masiva en milisegundos con alta tasa de inserción concurrente.

3. Método o propuesta de los autores

Los autores diseñan y presentan **Milvus**, una arquitectura de sistema gestor de bases de datos vectoriales desacoplada y nativa de la nube (*cloud-native*). El sistema separa formalmente la computación sin estado (*stateless query nodes*) del almacenamiento con estado, integrando múltiples algoritmos de búsqueda aproximada de vecinos más cercanos (ANN como HNSW, IVF-PQ y ANNOY) acelerados por GPU y SIMD, implementando además un modelo de consistencia sintonizable y un motor de inserción continua de datos mediante colas de mensajes (Apache Pulsar/Kafka).

4. Resultado principal

Milvus demostró procesar consultas por similitud sobre conjuntos de datos de más de mil millones de vectores (1 Billion scale) con latencias de consulta inferiores a 10 milisegundos y un rendimiento hasta tres órdenes de magnitud superior a las extensiones vectoriales añadidas sobre motores relacionales tradicionales, logrando escalabilidad elástica horizontal lineal en entornos distribuidos.

5. Relación explícita con la Unidad Temática I

Se relaciona directamente con el **Subtema 5: Tipos de bases de datos (Bases de datos vectoriales)** y el **Subtema 7: Bases de datos en flujos de trabajo de Inteligencia Artificial**. El artículo formaliza cómo la representación del conocimiento mediante vectores latentes (*embeddings*) exige un replanteamiento de los tipos de datos atómicos y de los índices en los SGBD modernos para servir a sistemas RAG y modelos multimodales.

6. Aporte para el proyecto del curso

Me proporciona una visión técnica rigurosa sobre cómo persistir y consultar vectores numéricos en escenarios donde el modelo entidad-relación se complementa con componentes de inteligencia artificial, comprendiendo la necesidad de desacoplar los metadatos relacionales (guardados en PostgreSQL) de las representaciones vectoriales densas.

Ficha Técnica 2: Optimización de Estructuras de Indexación mediante Inteligencia Artificial

1. Cita en formato APA 7.^a edición con DOI

Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018). The case for learned index structures. En *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)* (pp. 489–504). Association for Computing Machinery. <https://doi.org/10.1145/3183713.3196909>

2. Problema que aborda el artículo

Cuestiona la suposición de diseño que ha perdurado por más de cuatro décadas en las bases de datos: que las estructuras de indexación tradicionales (como árboles B+, tablas hash y filtros de Bloom) deben ser estructuras algorítmicas de propósito general que ignoran la distribución estadística subyacente de los datos que indexan.

3. Método o propuesta de los autores

Los autores proponen el paradigma de **Índices Aprendidos** (*Learned Index Structures*), demostrando matemáticamente que un índice de base de datos no es más que una función acumulada de distribución (CDF) de los datos que predice la posición de memoria de un registro a partir de su clave. Diseñan modelos jerárquicos de regresión y redes neuronales ligeras (*Recursive Model Index - RMI*) que sustituyen a los nodos y punteros de los árboles B+ convencionales.

4. Resultado principal

Los índices aprendidos demostraron reducir el consumo de memoria en hasta un **70%** y superar la velocidad de búsqueda de los árboles B+ tradicionales en un factor de hasta **3x** en operaciones de lectura, demostrando que los modelos estadísticos de aprendizaje automático pueden ejecutarse más rápido que el recorrido de ramificaciones de punteros en cachés modernas de CPU.

5. Relación explícita con la Unidad Temática I

Se relaciona con el **Subtema 3: Arquitectura ANSI-SPARC (Esquema Interno)** y el **Subtema 4: Módulos componentes de un SGBD (Gestor de Almacenamiento e Índices)**. El artículo transforma la concepción clásica del esquema interno, demostrando que la implementación física de punteros a bloques de disco puede sustituirse por funciones matemáticas aprendidas.

6. Aporte para el proyecto del curso

Como estudiante de Ingeniería en Inteligencia Artificial, me demuestra que las bases de datos y el aprendizaje automático no son disciplinas separadas: los principios de optimización interna de un motor de bases de datos pueden ser revolucionados mediante modelos estadísticos y algoritmos predictivos.

Ficha Técnica 3: Arquitectura Cloud-Native y Desacoplamiento Almacenamiento-Cómputo

1. Cita en formato APA 7.^a edición con DOI

Verbitski, A., Gupta, A., Saha, D., Brahmadesam, M., Gupta, K., Mittal, R., Krishnamurthy, S., Maurice, S., Kharatishvili, T., & Bao, X. (2018). Amazon Aurora: On avoiding distributed consensus for I/Os, commits, and membership changes. En *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)* (pp. 789–796). Association for Computing Machinery. <https://doi.org/10.1145/3183713.3190663>

2. Problema que aborda el artículo

Examina el cuello de botella tradicional de los SGBD relacionales en infraestructuras distribuidas en la nube, donde la replicación de bloques de disco completos y la sincronización síncrona de transacciones mediante protocolos de consenso distribuidos saturaban el ancho de banda de red y degradaban la latencia del commit.

3. Método o propuesta de los autores

Presentan el diseño arquitectónico de **Amazon Aurora**, cuyo principio rector es "*The Log is the Database*" (El registro es la base de datos). Desacoplan el nodo de procesamiento de consultas SQL del subsistema de almacenamiento persistente, delegando la reconstrucción de páginas de datos a una flota de almacenamiento distribuida en seis réplicas geográficas que solo intercambian registros de transacciones (*redologs*), eliminando el tráfico de páginas sucias (*dirty pages*) en la red.

4. Resultado principal

El sistema demostró un incremento de rendimiento transaccional de hasta **5 veces respecto a PostgreSQL nativo** y **3 veces respecto a MySQL nativo**, con alta disponibilidad y tolerancia a fallas de centros de datos completos sin degradación perceptible en los tiempos de confirmación (*commit*).

5. Relación explícita con la Unidad Temática I

Se vincula con el **Subtema 4: Independencia física de datos y Módulos componentes (Gestor de Transacciones y WAL)** y el **Subtema 6: Clasificación de SGBD (Centralizados vs. Distribuidos en la Nube)**. Ilustra cómo la independencia física permite alterar por completo el motor de almacenamiento subyacente sin modificar la interfaz relacional SQL ni el esquema conceptual percibido por las aplicaciones.

6. Aporte para el proyecto del curso

Me proporciona el marco conceptual necesario para comprender cómo los entornos modernos en contenedores y plataformas cloud gestionan la persistencia y la tolerancia a fallos mediante el desacoplamiento estricto de capas, principio que aplicamos en el `compose.yaml` al independizar el contenedor efímero del volumen con los datos.

Síntesis Comparativa de la Literatura Científica Consultada

Elementos Convergentes (Puntos en común)

Al analizar conjuntamente los tres trabajos seleccionados (Wang et al., 2021; Kraska et al., 2018; Verbitski et al., 2018), resalta un patrón arquitectónico común: **la ruptura del diseño monolítico tradicional de los SGBD**. Los tres artículos coinciden en que la arquitectura monolítica donde un único proceso gestiona simultáneamente el procesamiento de consultas, la indexación en memoria y la escritura directa en bloques de disco locales es insuficiente para las demandas computacionales contemporáneas. Tanto Milvus como Aurora optan por el desacoplamiento radical entre cómputo (*stateless processing*) y almacenamiento (*stateful persistence*), mientras que Kraska et al. atacan la rigidez de los algoritmos monolíticos de indexación. Asimismo, los tres artículos abordan el aprovechamiento exhaustivo del hardware moderno (procesamiento vectorial SIMD, núcleos masivos de GPU y redes de alta velocidad con baja latencia).

Divergencias Metodológicas

A pesar de sus metas de eficiencia, sus enfoques discrepan fundamentalmente en el plano epistemológico del dato y su abstracción:

* Mientras Verbitski et al. (2018) conservan intacto el modelo relacional clásico de Codd y el lenguaje SQL para proteger las inversiones de software existentes (modificando únicamente la plomería de red y el registro transaccional), Wang et al. (2021) abandonan la semántica relacional para concebir un modelo de datos centrado en espacios vectoriales euclidianos y métricas de distancia angular.

* Por su parte, Kraska et al. (2018) no proponen un nuevo sistema gestor completo, sino una disrupción algorítmica interna en la teoría de estructuras de datos discretas, reemplazando árboles jerárquicos deterministas por modelos estocásticos y redes neuronales continuas.

Problema Abierto Identificado

A partir de la lectura crítica de la literatura contemporánea, se identifica un problema abierto central en la frontera de las bases de datos y la inteligencia artificial: **el soporte unificado y transaccional ACID para datos multimodales y vectoriales**. En la actualidad, las organizaciones deben fragmentar su infraestructura: mantienen sus transacciones financieras en un RDBMS relacional (como PostgreSQL) y sincronizan mediante complejos pipelines de datos los embeddings hacia bases de datos vectoriales dedicadas (como Milvus).

Esta fragmentación introduce retrasos de sincronización (*sync lag*), riesgos de inconsistencia de datos y complejidades operativas severas. El reto científico abierto radica en diseñar motores de bases de datos híbridos capaces de mantener aislamiento ACID estricto y baja latencia de indexación vectorial en tiempo real sin requerir arquitecturas fragmentadas ni comprometer la independencia de datos formulada en la arquitectura ANSI-SPARC.